Reinforcement-Learning-Based Network Slicing and Resource Allocation for Multi-Access Edge Computing Networks

Shurui Jiang, Jun Zheng, *Senior Member, IEEE*, Feng Yan, *Member, IEEE*, and Shuyuan Zhao

***Abstract*—This paper considers the network slicing and resource allocation problem in a multi-access edge computing (MEC) network. A two-level slicing model is introduced for network slicing and the considered problem is formulated as a long-term optimization problem with an objective to maximize a weighted average of the successful processing ratio of users' computing tasks in each network slice instance by performing network slicing and resource allocation, while satisfying the service requirements of users' computing tasks. To solve the formulated problem, we decompose the problem into two sub-problems: periodic service-level slicing (SLS) and dynamic resource allocation (DRA), and propose a reinforcement-learning based network slicing and resource allocation (RL-S&A) algorithm, which consists of a QMIX-based SLS algorithm and a PPO-based DRA algorithm, to solve the two sub-problems, respectively. To reduce the training time of an optimal policy to perform periodic SLS, an MAML-based initialization algorithm is introduced to obtain a meta-policy to initialize the neural networks used in the QMIX-based SLS algorithm. Simulation results show that the proposed RL-S&A algorithm can achieve better performance than several benchmark algorithms in terms of the successful processing ratio of users' computing tasks in the network.**

***Index Terms*—MEC, network slicing, resource allocation, reinforcement learning.**

I. Introduction

MULTI-ACCESS edge computing (MEC) is a new computing technology which sinks computing, storage, and network control functions from a remote cloud center in a core network to the edge of the network, and thus is able to provide fast computing services in proximity of mobile users [1], [2]. In an MEC network, mobile users no longer need to offload their computing tasks to a remote cloud center but to an edge node (or server), which can largely reduce not only the service delay of a task but also the bandwidth required to transmit a computing task to a remote cloud center.

An MEC network is expected to provide low-delay and high-reliability computing services for mobile users. Due to the limitation of network resources in an MEC network, resource allocation becomes a critical issue in achieving low-delay and high-reliability performance. To address this issue, network slicing (NS) based on network function virtualization (NFV) and software defined networks (SDNs) can be introduced to improve the efficiency of resource allocation and utilization of network resources [3], [4], [5]. Network slicing abstracts physical resources into virtual resources through NFV, and builds end-to-end logical networks based on specific network functions and access network technologies. A network slicing instance is able to adjust resource allocation according to users' dynamic demands, and thus provide optimal resource allocation to meet users' different service requirements. Compared with network slicing for other types of networks, the challenge of network slicing for an MEC network is to face diverse access network technologies and satisfy more diverse service requirements of mobile users. To address this challenge, it is necessary to introduce new technologies to improve network performance.

Machine learning (ML) has demonstrated its capability in improving network performance and service quality through intelligently managing network resources. For many optimization problems, it is difficult to find an optimal solution using classical methods, while ML is more effective in this respect to a large extent, especially in a dynamic network environment. Therefore, it is necessary and interesting to introduce ML in network slicing and resource allocation for an MEC network, and study ML-based network slicing and resource allocation algorithms.

In this paper, we consider the network slicing and resource allocation (S&A) problem in an MEC network. We introduce a two-level slicing model and formulate the problem as a long-term optimization problem with an objective to maximize a weighted average of the successful processing ratios of users' computing tasks in each network slice instance by performing network slicing and resource allocation, while satisfying the service requirements of users'

Manuscript received 30 July 2023; revised 29 November 2023; accepted 6 December 2023. Date of publication 13 December 2023; date of current version 7 June 2024. This work is supported by Open Research Projects of Zhejiang Lab (No. 2021LC0AB02), the Fundamental Research Funds for the Central Universities (No. 2242022k60001), and the Key Research & Developement Plan of Jiangsu Province (Grant No. BE2020084-2). The associate editor coordinating the review of this article and approving it for publication was Q. J. Ye. *(Corresponding author: Jun Zheng.)*

Shurui Jiang is with the National Mobile Communications Research Laboratory, School of Information Science and Engineering, Frontiers Science Center for Mobile Information Communication and Security, Southeast University, Nanjing 210096, China (e-mail: srjiang@seu.edu.cn).

Jun Zheng is with the National Mobile Communications Research Laboratory, School of Information Science and Engineering, Frontiers Science Center for Mobile Information Communication and Security, Southeast University, Nanjing 210096, China, also with Zhejiang Laboratory, Hangzhou 311121, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: junzheng@seu.edu.cn).

Feng Yan is with the National Mobile Communications Research Laboratory, School of Information Science and Engineering, Frontiers Science Center for Mobile Information Communication and Security, Southeast University, Nanjing 210096, China, and also with Zhejiang Laboratory, Hangzhou 311121, China (e-mail: feng.yan@seu.edu.cn).

Shuyuan Zhao is with Zhejiang Laboratory, Hangzhou 311121, China (e-mail: zhaosy@zhejianglab.com).

Digital Object Identifier 10.1109/TCCN.2023.3342441

2332-7731 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

computing tasks. To solve the formulated problem, we decompose the problem into two sub-problems: periodic service-level slicing (SLS) and dynamic resource allocation (DRA), and propose a reinforcement-learning (RL) based network slicing and resource allocation (RL-S&A) algorithm, which consists of a QMIX-based SLS algorithm and a proximal policy optimization (PPO) based DRA algorithm to solve the two sub-problems, respectively. To reduce the training time of an optimal policy to perform periodic SLS, an MAML-based initialization algorithm is introduced to obtain a meta-policy to initialize the neural networks used in the QMIX-based algorithm. Simulation results are presented to evaluate the performance of the proposed RL-S&A algorithm in terms of the successful processing ratio of users' computing tasks in the network.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the system model and formulates the problem considered in this paper. Section IV presents the proposed RL-S&A algorithm. Section V presents simulation results to evaluate the performance of the proposed algorithm. Section VI concludes this paper.

II. Related Work

Network slicing and resource allocation have recently been extensively studied in literature [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22]. In the context of resource allocation, Tran and Pompili [6] considered a joint task offloading and resource allocation problem to maximize a user's task offloading revenue in a multi-cell wireless network, and used the convex optimization method and a new heuristic algorithm to solve the problem. Guo et al. [7] studied a computing offloading problem considering the communication and computing dynamics in an MEC system, and designed an online computing offloading mechanism based on a multi-arm gambling machine framework to minimize the average expected task execution delay with an average energy consumption constraint. Ale et al. [8] studied a computation offloading problem with the objective to maximize the number of tasks completed before deadlines and minimize energy consumption. An end-to-end deep reinforcement learning algorithm was proposed to select the best edge server for offloading and allocating the computing resources. Wang et al. [9] studied an offloading and computing resource allocation problem in an MEC system to meet mobile devices' requirements and proposed a resource allocation algorithm based on deep reinforcement learning, which adaptively allocates computing and network resources, and balances the use of the resources. Wang et al. [10] studied the latency minimization problem in a NOMA-MEC network and proposed an iterative two-user NOMA scheme for task offloading. Sharif et al. [11] studied a resource utilization optimization problem with limited resources and proposed an adaptive resource allocation mechanism for efficient resource utilization in an EC paradigm.

In network slicing, both slicing frameworks and resource slicing algorithms are critical for improving the performance and service quality of a network. In the context of slicing frameworks, Ye et al. [12] proposed a RAN slicing framework for performing resource slicing among BSs in a cloud-enabled autonomous vehicular network. Ye et al. [13] proposed a two-tier network slicing framework, where the total bandwidth resources are firstly sliced among BSs, and then split and allocated to two types of services. Ndikumana et al. [14] proposed a RAN slicing framework, which first slices resource blocks to tenants and then maps resource blocks to Open Distributed Units (O-DUs). Awad Abdellatif et al. [15] proposed an optimized and intelligent network slicing framework to maintain a high performance of network operation, and support diverse and heterogeneous services through selection of radio access points, VNF placement and data routing. Zhang et al. [16] proposed a new performance-guaranteed slicing strategy with a soft and hard hybrid slicing setting using deep reinforcement learning.

In the context of slicing algorithms, Feng et al. [17] proposed a new framework of network slicing for an MEC system and proposed an algorithm using Lyapunov optimization technique to maximize the average revenue of an operator. Hua et al. [18] studied the demand-aware allocation problem in network slicing, and proposed a deep distributed Q network based on a generative adversarial network to optimize the quality-of-experience satisfaction ratio and spectrum efficiency. Chen et al. [19] studied the computing offloading problem in an ultra-dense sliced RAN with the objective to maximize long-term utility performance and proposed a strategy computation offloading algorithm based on double deep-Q-network to learn an optimal strategy. Dong et al. [20] introduced a joint network slicing and routing mechanism, and proposed a graph convolutional network (GCN) driven multi-task deep reinforcement learning (DRL) algorithm to solve the resource allocation problem. Sebakara et al. [21] studied an interdependent resource allocation problem and proposed a slice-to-node access factor (SNAF) based end-to-end slice resource provisioning scheme and a DRL-based real-time resource allocation algorithm. Mohammed et al. [22] studied a slice admission decision problem and proposed a multi-resource Fog-RAN slicing algorithm that maximizes network resource utilization and satisfies economic properties.

Compared with the extensive studies on network slicing and resource allocation for MEC networks, this work proposes a new two-level slicing model which consists of network-level slicing and service-level slicing, and studies both periodic network slicing and dynamic resource allocation using an RL method. Compared with existing network slicing models, the proposed two-level slicing model has better flexibility for resource management.

III. System Model and Problem Formulation

In this section, we describe the network model and formulate the S&A problem considered in this paper.

A. Network Model

Consider an MEC network with multiple edge nodes and a number of mobile users, as shown in Fig. 1. Each mobile user can generate different types of computing tasks with different

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

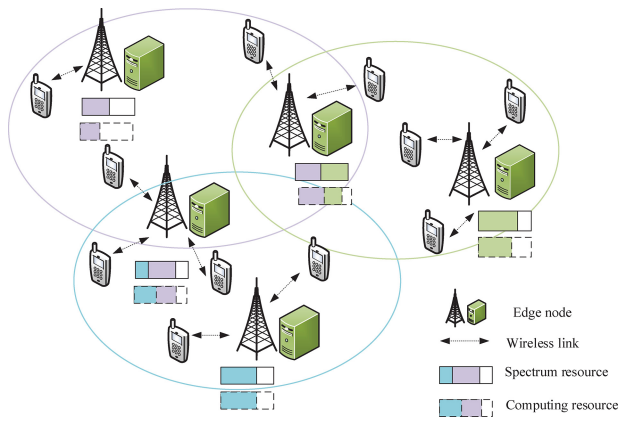

Fig. 1. Network model.

service requirements. Each edge node is composed of a base station (BS) and an associated edge server (ES), which provide underlying physical resources. NFV is used to abstract the physical resources into virtual resources and SDN is used to realize a flexible configuration of network functions, which enables the network to use common physical resources to provide services for the users.

Assume that there are M access network technologies in the MEC network and thus the network need to provide M logical access networks for its mobile users based on all its physical resources, which are denoted by $\boldsymbol{AN} = \{AN_m, m = 1, 2, \ldots, M\}$. There are a set of N edge nodes in the MEC network, which is denoted by $\boldsymbol{E} = \{E_n, n = 1, 2, \ldots, N\}$. Correspondingly, there are a set of N BSs and a set of N associated ESs in the network, which are denoted by $\boldsymbol{BS} = \{BS_n, n = 1, 2, \ldots, N\}$, and $\boldsymbol{ES} = \{ES_n, n = 1, 2, \ldots, N\}$, respectively. Each BS has N_B^n orthogonal spectrum resource blocks (SRBs) and the size of each SRB is equal to B. Each ES has N_C^n computing resource blocks (CRBs) and the size of each CRB is equal to F. The total amount of physical resources in the network include the SRBs and CRBs in all BSs and ESs. Each user can generate L types of computing tasks with different service delay requirements. A user typically offloads a computing task to the nearest edge node for processing and each computing task can be offloaded to only one edge node. Moreover, all edge nodes in the network can exchange information with each other.

B. Slicing Model

To efficiently utilize the spectrum and computing resources in the network, we introduce a two-level slicing model: network-level slicing (NLS) and service-level slicing (SLS). At the network level, spectrum and computing resources in the network are sliced for different logical access networks on a periodic basis according to the current service demand of each logical access network, which is characterized by $q_{ns}^m(t) = (B_m(t), F_m(t), S_m(t))(m = 1, 2, \ldots, M)$, where $B_m(t)$, $F_m(t)$, and $S_m(t)$ denote the demand for spectrum resources, the demand for computing resources, and the coverage (i.e., a set of involved BSs) of the m-th logical access network, respectively. Assume that there is a virtual controller in the network, which dynamically collects network status information from each edge node and performs NLS. After NLS is performed, a set of network-slice instances are created. We use $\boldsymbol{I}_{ns} = \{I_{ns}^m, m = 1, 2, \ldots, M\}$ to denote the set of created network-slice instances, where I_{ns}^m denotes the m-th network-slice instance.

In each network-slice instance, spectrum and computing resources are further sliced at the service level for different types of users' computing tasks on a periodic basis according to the current demand of each type of computing tasks, which is characterized by $q_{ss}^l(t) = (B_l(t), F_l(t), S_l(t))$ $(l = 1, 2, \ldots, L)$, where $B_l(t)$, $F_l(t)$, and $S_l(t)$ denote the demand for spectrum resources, the demand for computing resources, and the service requirement of type-l computing tasks, respectively. Assume that there is a virtual controller in each network-slice instance to perform SLS. After SLS is performed, a set of service-slice instances are created. We use $\boldsymbol{I}_{ss}^m = \{I_{ss}^{l,m}, l = 1, 2, \ldots, L\}$ to denote the set of created service-slice instances in the m-th network-slice instance, where $I_{ss}^{l,m}$ denotes the l-th service-slice instance.

Usually, NLS and SLS are performed at different time granularities. NLS is performed in a relatively long period, while SLS is more dynamic and is usually performed periodically.

C. Resource Allocation

In each service-slice instance $I_{ss}^{l,m}$, a user's computing task is dynamically offloaded to a nearby BS for processing. Once a user generates a type-l computing task $u_k^l(t)$, it will first transmit a request to the nearest BS which can provide the service for the computing task. The BS will allocate SRBs for the computing task and the data of the computing task will be transmitted to the server associated with the BS. The server will then allocate CRBs to the computing task and process the computing task. After the processing is completed, a processing result will be transmitted back to the user requesting for the service.

D. Delay Analysis

A user's computing task is dynamically offloaded to a slice instance for processing. Consider a typical user's computing task $u_k^l(t)$, i.e., the k-th computing task of type l. It is characterized by a tuple $u_k^l(t) = (L_k(t), X_k(t), \tau_{l,max})$, where $L_k(t)$, $X_k(t)$, and $\tau_{l,max}$ denote the data size, the computation intensity, and the maximum tolerant service delay of $u_k^l(t)$, respectively. Define $x_n^k(t) \in \{0, 1\}$ as a server selection indicator for $u_k^l(t)$, where $x_n^k(t) = 1$ indicates that $u_k^l(t)$ is offloaded to edge server ES_n; otherwise, $x_n^k(t) = 0$. Since a user's computing task can only be offloaded to one edge server for processing, the following condition should be satisfied:

$$\sum_{n=1}^{N} x_n^k(t) = 1. \tag{1}$$

If service slice instance $I_{ss}^{l,m}$ provides the service for $u_k^l(t)$, and the number of SRBs allocated to $u_k^l(t)$ on the n-th BS

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

in slice instance $I_{ss}^{l,m}$ is $B_{l,m,n}^{k}(t)$, according to Shannon's formula, the transmission rate of $u_k^l(t)$ is given by

$$r_k(t) = \sum_{n=1}^{N} x_n^k(t) B B_{l,m,n}^k(t) \log_2\left[1 + \frac{p_k(t) h_{k,n}(t)}{\sigma^2}\right], \quad (2)$$

where $p_k(t)$ denotes the transmission power of $u_k^l(t)$, $h_{k,n}(t)$ denotes the channel gain between the user generating $u_k^l(t)$ and the n-th BS, and σ^2 denotes the noise power on the channel. Thus, the transmission delay of $u_k^l(t)$ is given by

$$\tau_{k,tran}(t) = \frac{L_k(t)}{r_k(t)}. \quad (3)$$

If the number of CRBs allocated to $u_k^l(t)$ on the n-th edge server in slice instance $I_{ss}^{l,m}$ is $F_{l,m,n}^k(t)$, the computation delay of $u_k^l(t)$ is given by

$$\tau_{k,comp}(t) = \frac{L_k(t) X_k(t)}{F \times F_{l,m,n}^k(t)}. \quad (4)$$

If a slice instance fails to provide sufficient spectrum resources or computing resources for $u_k^l(t)$, $u_k^l(t)$ will enter a transmission queue or a computation queue to wait for transmission or processing. Therefore, the service delay of $u_k^l(t)$ includes transmission delay, computation delay, queuing delay for transmission, and queuing delay for computation, i.e.,

$$\begin{aligned}\tau_k(t) = {} & \tau_{k,tran}(t) + \tau_{k,comp}(t) \\ & + \tau_{k,tranq}(t) + \tau_{k,compq}(t),\end{aligned} \quad (5)$$

where $\tau_{k,tranq}(t)$ denotes the queuing delay before transmission, which is the time between the instant that $u_k^l(t)$ is generated and that $u_k^l(t)$ starts to be transmitted, and $\tau_{k,compq}(t)$ denotes the queuing delay before processing, which is the time between the instant $u_k^l(t)$ is successfully transmitted and that $u_k^l(t)$ starts to be processed. The readers are referred to [23] for a discussion of the queuing delays.

If the service delay of $u_k^l(t)$ is smaller than $\tau_{l,max}$, $u_k^l(t)$ is considered successfully processed. Let $U_{l,m,n}^s(t)$ denote the number of users' computing tasks processed successfully by the n-th edge server in slice instance $I_{ss}^{l,m}$ during a period and $U_{l,m,n}(t)$ denote the total number of users' computing tasks processed by the n-th edge server in slice instance $I_{ss}^{l,m}$ during a period. The successful processing ratio of users' computing tasks by the n-th edge server in slice instance $I_{ss}^{l,m}$ is given by

$$\eta_{l,m,n}(t) = \frac{U_{l,m,n}^s(t)}{U_{l,m,n}(t)}. \quad (6)$$

Thus, we can obtain the successful processing ratio in slice instance I_{ns}^m and that in slice instance $I_{ss}^{l,m}$, i.e., $\eta_m(t)$ and $\eta_{l,m}(t)$, i.e.,

$$\eta_m(t) = \frac{U_m^s(t)}{U_m(t)}, \quad (7)$$

$$\eta_{l,m}(t) = \frac{U_{l,m}^s(t)}{U_{l,m}(t)}, \quad (8)$$

where $U_m^s(t)$ denotes the number of users' computing tasks processed successfully in slice instance I_{ns}^m during a period, $U_m(t)$ denotes the total number of users' computing tasks processed in slice instance I_{ns}^m during a period, $U_{l,m}^s(t)$ denotes the number of users' computing tasks processed successfully in slice instance $I_{ss}^{l,m}$ during a period and $U_{l,m}(t)$ denotes the total number of users' computing tasks processed in slice instance $I_{ss}^{l,m}$ during a period. Thus, a weighted average of the successful processing ratios of users' computing tasks in each network slice instance is given by

$$\eta_{avg}(t) = \alpha_1 \eta_1(t) + \alpha_2 \eta_2(t) + \cdots + \alpha_M \eta_M(t), \quad (9)$$

where $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \ldots, \alpha_M](0 < \alpha_m < 1, \alpha_1 + \alpha_2 + \cdots + \alpha_M = 1)$ denotes a vector of weight coefficients for a set of given M network slice instances.

E. Problem Formulation

We consider the S&A problem in the MEC network described in Fig. 1. Given the total amount of spectrum resources and computing resources in the network, the objective of the S&A problem is to maximize the weighted average of the successful processing ratios of users' computing tasks in each network slice instance by performing network slicing and resource allocation, while satisfying the service requirements of users' computing tasks. Since NLS is typically performed by the virtual controller in a relatively long period, we assume that M network-slice instances are given, and focus on the study of periodic SLS for each type of computing tasks in each network-slice instance and DRA for each computing task. Thus, the S&A problem can be formulated as

P: Objective:

$$\max_{\boldsymbol{B},\boldsymbol{F},\boldsymbol{B}^k,\boldsymbol{F}^k} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \eta_{avg}(t), \quad (10)$$

subject to

$$\tau_k(t) \le \tau_{l,max}, \quad k = 1, 2, \ldots, U_{l,m,n}, l = 1, \ldots, L, \quad m = 1, \ldots, M, n = 1, \ldots, N \quad (10a)$$

$$\sum_{k=1}^{K(t)} B_{l,m,n}^k(t) \le B_{l,m,n}(t), \quad l = 1, 2, \ldots, L, \quad n = 1, 2, \ldots, N, \quad m = 1, 2, \ldots, M \quad (10b)$$

$$\sum_{k=1}^{K(t)} F_{l,m,n}^k(t) \le F_{l,m,n}(t), \quad l = 1, 2, \ldots, \quad L, n = 1, 2, \ldots, N, \quad m = 1, 2, \ldots, M \quad (10c)$$

$$\sum_{m=1}^{M} B_{m,n}(t) \le N_B^n, n = 1, 2, \ldots, N, \quad (10d)$$

$$\sum_{m=1}^{M} F_{m,n}(t) \le N_F^n, n = 1, 2, \ldots, N, \quad (10e)$$

$$\sum_{n=1}^{N} B_{m,n}(t) = B_m, m = 1, 2, \ldots, M, \quad (10f)$$

$$\sum_{n=1}^{N} F_{m,n}(t) = F_m, m = 1, 2, \ldots, M, \quad (10g)$$

where $\boldsymbol{B} = \{B_{l,m,n}(t) | l = 1, 2., \ldots, L, m = 1, 2, \ldots, M, n = 1, 2, \ldots, N, t = 1, 2, \ldots, T\}, \boldsymbol{F} = \{F_{l,m,n}(t) | l = 1, 2, \ldots, L, m = 1, 2, \ldots, M, n = 1, 2, \ldots, N, t = 1, 2, \ldots, T\}, \boldsymbol{B}^k = \{B_{l,m,n}^k(t) | l =$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

$1, 2., \ldots, L, m = 1, 2, \ldots, M, n = 1, 2, \ldots, N, k = 1, 2, \ldots, U_{l,m,n}, t = 1, 2, \ldots, T\}$, $\boldsymbol{F}^k = \{F^k_{l,m,n}(t)|l = 1, 2., \ldots, L, m = 1, 2, \ldots, M, n = 1, 2, \ldots, N, k = 1, 2, \ldots, U_{l,m,n}, t = 1, 2, \ldots, T\}$ and $K(t)$ denotes the number of users' computing tasks in slice instance $I^{l.m}_{ss}$ at time t. Eq. (10a) specifies that the service delay of a type-l computing task should not be larger than the maximum tolerant service delay, Eq. (10b)/Eq. (10c) ensures that the total number of SRBs/CRBs allocated to users' computing tasks arriving in slice instance $I^{l.m}_{ss}$ do not exceed the total number of SRBs/CRBs of slice instance $I^{l.m}_{ss}$, Eq. (10d)/Eq. (10e) ensures that the total number of SRBs/CRBs allocated to all M network slice instances in a BS do not exceed the total number of SRBs/CRBs in the BS, Eq. (10f)/Eq. (10g) ensure that the number of SRBs/CRBs sliced to all N BSs/ESs in slice instance I^m_{ns} is equal to the number of SRBs/CRBs allocated to slice instance I^m_{ns}.

IV. Reinforcement-Learning Based Network Slicing and Resource Allocation Algorithm

This section presents the proposed RL-S&A algorithm for solving the formulated problem.

A. Overview

Since SLS and DRA are performed at different time granularities, it is hard to solve the overall formulated problem using a traditional algorithm. To solve the problem, we decompose the problem into two sub-problems: SLS problem (SP1) and DRA problem (SP2). The objective of SP1 is to maximize the weighted average of the successful processing ratios of users' computing tasks in each network-slice instance by periodically performing SLS in each network-slice instance, i.e.,

SP1: objective

$$\max_{\boldsymbol{B},\boldsymbol{F}} \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \eta_{avg}(t), \tag{11}$$

subject to Eq. (10d) $\sim$ Eq. (10g).

The objective of SP2 is to maximize the successful processing ratio of users' computing tasks in an edge node involved with a service-level slice instance by performing DRA for each computing task, i.e.,

SP2: objective

$$\max_{\boldsymbol{B}^k,\boldsymbol{F}^k} \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \eta_{l,m,n}(t), \tag{12}$$

subject to Eq. (10a) $\sim$ Eq. (10c).

Correspondingly, we propose a QMIX-based SLS algorithm and a PPO-based DRA algorithm to solve SP1 and SP2, respectively. The former is used to perform SLS in each network-slice instance while the latter is used to perform DRA in each service-slice instance. Therefore, the proposed RL-S&A algorithm consists of these two algorithms. In the QMIX-based SLS algorithm, a group of agent networks, mixing network, and hyper-networks are trained periodically using previous experiences. Once it is time for slicing, the trained agent networks are used to perform SLS. In the PPO-based DRA algorithm, a group of actor networks and critic networks are trained more frequently than a group of neural networks used in SLS. Once a user's computing task arrives in a service-slice instance, the trained neutral networks are used to allocate resources for the computing task. Since DRA is performed during each period of SLS, the QMIX-based SLS algorithm and the PPO-based DRA algorithm depend on each other. The DRA for computing tasks using the PPO-based DRA algorithm would affect the successful processing ratio of computing tasks in each service-level instance. Since the successful processing ratio in each service-slice instance is part of the inputs to the neural networks used in the QMIX-based SLS algorithm, it would further affect the SLS in each network-slice instance.

B. QMIX-Based SLS Algorithm

SP1 is a long-term optimization problem. It can be considered as a Markov decision problem [24] because the network environment at time $t+1$ only depends on the environment and the SLS at time t. Thus, we propose a QMIX-based SLS algorithm to solve the problem.

1) QMIX Algorithm: A QMIX algorithm [25] is a multi-agent reinforcement learning (MARL) algorithm, which aims to learn a joint Q-function $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$, where $\boldsymbol{\tau}(t)$ denotes a joint action observation history and $\boldsymbol{a}(t)$ denotes a joint action vector. It uses a centralized training and decentralized execution mode, in which a central controller trains a group of agent networks with global information on an environment in a centralized manner, and a group of agents choose and execute actions based on local information on the environment in a decentralized manner. The decentralized execution manner relieves the problem that the joint action space increases exponentially with the number of agents, which often makes traditional single-agent RL algorithms unusable.

At each step of a QMIX algorithm, an agent first obtains a local observation on the environment $o_j(t)$, then chooses an action $a_j(t)$ according to $o_j(t)$, executes the chosen action, and obtains a reward $r(t)$, where j denotes the index of an agent. After the execution of the chosen action, the environment will be updated and the agent will obtain a new local observation at the next step $o_j(t+1)$. The central controller will collect the new observations of all agents and store the experience $(\boldsymbol{s}(t), \boldsymbol{a}(t), r(t), \boldsymbol{s}(t+1))$ into an experience pool, in which $\boldsymbol{s}(t)/\boldsymbol{s}(t+1)$ is a vector containing the observations of all agents at step $t/t+1$. Then, it will sample a batch of experiences from the experience pool to train all agent networks used in the QMIX algorithm.

A QMIX algorithm uses a set of neural networks, including a set of agent networks, a mixing network, and a set of hyper-networks, to learn a joint Q-function $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$, which are described respectively as follows:

(1) Agent Network: An agent network is a network which is used by each agent to fit a Q-function and perform decentralized execution. An agent network uses the structure of a deep recurrent Q-network (DRQN), which includes an

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

evaluation network and a target network. The inputs of an agent network include the current observation $o_j(t)$ and the previous action $a_j(t-1)$. The output of the network is a Q-value, $Q_j(\tau_j(t), a_j(t))$.

(2) Mixing Network: A mixing network is a fully connected network which receives the output value $Q_j(\tau_j(t), a_j(t))$ of each agent as an input, and outputs the value of a joint Q-function, $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$. It is used to perform a nonlinear mapping between the value of a joint Q-function and the value of each agent's Q-function. To structurally ensure that the joint Q-value is monotonic on each individual Q-value, i.e., the consistency between the centralized training and the decentralized execution, it is necessary to introduce a constraint between the joint Q-function $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$ and each agent's Q-function $Q_j(\tau_j(t), a_j(t))$, i.e.,

$$\frac{\partial Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{\alpha}(t))}{\partial Q_j\big(\tau_j(t), a_j(t)\big)} \geq 0. \tag{13}$$

(3) Hyper-Network: A hyper-network is a network which is used to generate the weights of a mixing network. Its inputs are the local observations of all agents, and its outputs are the weights of the mixing network.

Similar to a deep Q-network (DQN), a QMIX algorithm trains the parameters of all its agent networks by minimizing its loss function [25], which is defined as

$$L(\boldsymbol{\theta}_e) = \sum_{i=1}^{b} \Big[\big(y_i^{tot} - Q_{tot}(\boldsymbol{\tau}, \boldsymbol{a}, s; \boldsymbol{\theta}_e)\big)^2\Big], \tag{14}$$

$$y_i^{tot} = r + \gamma \max_{u'} Q_{tot}\big(\boldsymbol{\tau}', \boldsymbol{a}', s'; \boldsymbol{\theta}_t\big), \tag{15}$$

where b represents the number of experiences sampled from the experience pool of the QMIX algorithm, $\boldsymbol{\tau}$ denotes a joint action-observation history of all evaluation networks used in the QMIX algorithm, $\boldsymbol{a}$ denotes a joint action vector of all the evaluation networks, s denotes the state of the QMIX algorithm, r denotes the reward of an action, γ denotes a discount factor, $\boldsymbol{\theta}_e/\boldsymbol{\theta}_t$ denotes a set of parameters of all evaluation/target networks used in the QMIX algorithm, $\boldsymbol{\tau}'$ denotes the joint action-observation history of all the target networks corresponding to the maximum $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$, $\boldsymbol{a}'$ denotes a joint action vector of all the target networks corresponding to the maximum $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$, and s' denotes the state of the QMIX algorithm corresponding to the maximum value of $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$.

As discussed earlier, the MEC network provides M logical access networks, which correspond to M network-slice instances. The environment of the network can be reflected by the number of current users' computing tasks and the current status of spectrum and computing resources in each network-slice instance. The virtual controller of each network-slice instance I_{ns}^m can only observe its local environment and needs to perform SLS in its own network-slice instance. At this point, the network model can be considered as a decentralized execution model. On the other hand, each network-slice instance I_{ns}^m depends on each other. Since the total amount of spectrum and computing resources are limited in all edge nodes and all M logical access networks share the same amount of resources, it is necessary to use a centralized training mode to perform SLS in order to obtain an optimal slicing policy. Since a QMIX algorithm uses a centralized training and decentralized execution mode to obtain an optimal policy, it is suitable for being used to solve SP1. For the constraint of the QMIX algorithm shown in Eq. (13), the elements in $\boldsymbol{\alpha}$ are all positive values, which ensures that the joint Q-value is monotonic on each individual Q-value.

As an MARL algorithm, the QMIX algorithm can improve the relationship between $Q_j(\tau_j(t), a_j(t))$ and $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$ compared with the value-decomposition network (VDN) algorithm [26], which is a classical MARL algorithm. In the VDN algorithm, $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$ is a sum of $Q_j(\tau_j(t), a_j(t))$, which adds a constraint to the applicability of the VDN algorithm. In the QMIX algorithm, the relationship between $Q_j(\tau_j(t), a_j(t))$ and $Q_{tot}(\boldsymbol{\tau}(t), \boldsymbol{a}(t))$ is learned through neutral network training, which has a larger application scope and better performance.

2) QMIX-Based SLS Algorithm: The QMIX-based SLS algorithm uses a QMIX algorithm to perform periodic SLS in each network-slice instance. In the algorithm, the agent, state, action and reward function are defined as follows:

Agent: An agent is a virtual controller which performs periodic SLS in each network-slice instance I_{ns}^m.

State: The state of the algorithm is represented by a set of local observations obtained by all agents in the algorithm. The local observation of each agent is represented by

$$o_m(t) = \Big\{ B_{l,m,n}(t), F_{l,m,n}(t), \eta_{l,m,n}(t), U_{l,m,n}(t) \,|n = 1, 2, \ldots, N_m, l = 1, 2, \ldots, L \Big\}, \tag{16}$$

where $B_{l,m,n}(t)$ denotes the number of SRBs on the n-th BS in slice instance $I_{ss}^{l,m}$, $F_{l,m,n}(t)$ denotes the number of CRBs on the n-th ES in slice instance $I_{ss}^{l,m}$, $\eta_{l,m,n}(t)$ denotes the successful processing ratio on the n-th edge node in slice instance $I_{ss}^{l,m}$, $U_{l,m,n}(t)$ denotes the number of users' computing tasks per unit time on the n-th edge node in slice instance $I_{ss}^{l,m}$, and N_m denotes the number of BSs/ESs in network-slice instance I_{ns}^m. Based on the local observation of each agent, the state of the QMIX algorithm is defined as

$$\boldsymbol{s}(t) = \{o_m(t) | m = 1, 2, \ldots, M\}. \tag{17}$$

Action: The action of an agent is to perform SLS in each network-slice instance I_{ns}^m. Specifically, an agent can choose to increase, decrease or maintain the number of SRBs or CRBs on each BS in its I_{ns}^m. The action space of each agent can be represented by

$$a_m(t) = \Big\{ V_{l,m,n}(t), W_{l,m,n}(t) | n = 1, \ldots, N_m, l = 1, \ldots, L, \sum_{n=1}^{N} B_{m,n}(t) = B_m, \sum_{n=1}^{N} F_{m,n}(t) = F_m \Big\}, \tag{18}$$

where $V_{l,m,n}(t) = 1$ and $W_{l,m,n}(t) = 1$ indicate an increase of SRBs and CRBs on the n-th edge server in slice instance $I_{ss}^{l,m}$, respectively; $V_{l,m,n}(t) = -1$ and $W_{l,m,n}(t) = -1$ indicate a decrease of SRBs and CRBs, respectively; and

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

$V_{l,m,n}(t) = 0$ and $W_{l,m,n}(t) = 0$ indicate no change of SRBs and CRBs, respectively. The number of resource blocks increased or decreased in each action is set to a fixed value. The joint action vector of the QMIX algorithm is composed of each agent's action, which can be represented by

$$\boldsymbol{a}(t) = \{a_m(t) | m = 1, 2, \ldots, M\}. \tag{19}$$

Reward function: Since the objective of the formulated problem is to maximize a weighted average of the successful processing ratios in a long term, we design the reward function as a form similar to the objective, which is defined in Eq. (20), shown at the bottom of the page. In Eq. (20), $r_m(t)(m = 1, 2, \ldots, M)$ is a utility function reflecting the relationship between a reward and the average successful processing ratio of users' computing tasks in network-slice instance I_{ns}^m. According to Eq. (20), if the total amount of spectrum resources mapped to each BS in all I_{ns}^m does not exceed the total amount of spectrum resources of each BS, and the total amount of computing resources mapped to each BS in all I_{ns}^m does not exceed the total amount of computing resources of each BS, the reward value is a weighted average of $r_m(t)$; otherwise, the reword value is set to -1.

As the average successful processing ratio $\eta_m(t)$ usually does not follow a uniform distribution in a real network, its value is usually concentrated in a relatively small range. To more obviously reflect the impact of $\eta_m(t)$ on the reward value, it is necessary to appropriately extend the value range of $\eta_m(t)$. Thus, we define the utility function for $\eta_m(t)$ as

$$r_m(t) = \frac{1}{1 + e^{-\beta_1[\eta_m(t) - \beta_2]}}, \tag{21}$$

where β_1 and $\beta_2 (\beta_1 > 0, 0 < \beta_2 < 1)$ denotes two coefficients. β_1 determines the changing rate of $r_m(t)$ with $\eta_m(t)$ and β_2 determines the value of $\eta_m(t)$ when $r_m(t) = 0.5$. Thus, the utility function can map the average successful processing ratio to a relatively larger value range (0, 1) through appropriate coefficient selection. .

Fig. 2 illustrates the structure of the QMIX-based SLS algorithm, in which FC denotes a fully connected layer, GRU denotes a gate recurrent unit and π denotes a ε-greedy strategy. The pseudo codes of the QMIX-based SLS algorithm are described in Algorithm 1, where N_{epis}^Q denotes the number of episodes, N_{step}^Q denotes the number of steps in an episode and p denotes the update cycle of a target network.

3) MAML-Based Initialization Algorithm: Given a weight vector, the QMIX-based SLS algorithm is used to train an optimal policy to perform periodic SLS to achieve the objective with the given weight vector. When the weight vector changes, the algorithm needs to retrain an optimal policy to perform periodic SLS, which needs to take much training time. To reduce the training time, an MAML-based initialization algorithm can be introduced to obtain a meta-policy, i.e., a set of optimal initialization parameters, to initialize the agent networks, mixing networks, and hyper-networks used in the QMIX-based algorithm.

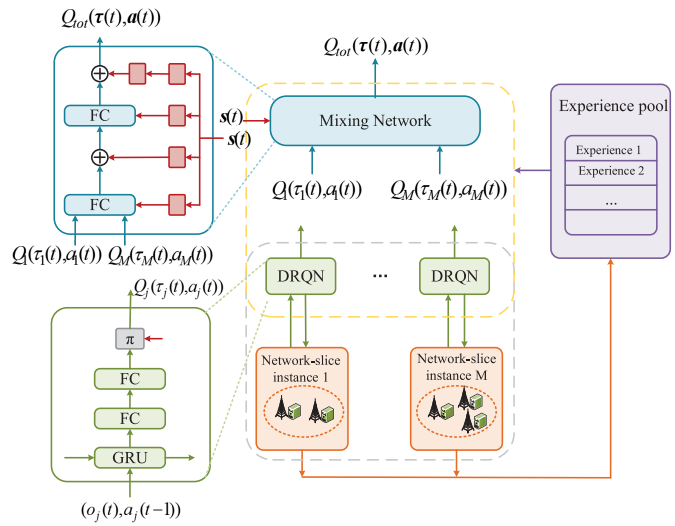

Fig. 2. Illustration of the QMIX-based SLS algorithm.

Algorithm 1 QMIX-Based SLS Algorithm

Initialize evaluation networks, target networks, a mixing network, hyper-networks;
for $t = 1$ to N_{epis}^Q
  Initialize the environment;
  for $step = 1$ to N_{step}^Q
    Obtain the observation of each agent $o_m(t)$;
    Choose actions $\boldsymbol{a}(t)$ for agents according to an ε-greedy strategy;
    Execute actions $\boldsymbol{a}(t)$;
    Obtain a reward $r(t)$ and update the environment;
    Store the experience $(\boldsymbol{s}(t), \boldsymbol{a}(t), r(t), \boldsymbol{s}(t+1))$ into an experience pool;
  if the experience pool is full
    Sample a batch of experiences from the experience pool randomly;
    Update the parameters of the evaluation networks according to Eq. (14);
    if $t \bmod p = 0$
      Update the target networks;

a) Meta-policy and MAML algorithm: A meta-policy [27] is a fast adaptive policy which can fast optimize an invisible objective constructed by a new weight vector and can be used as an initial optimal policy to effectively train different objectives with different weight vectors. Without loss of generality, a model-agnostic meta-learning (MAML) algorithm can be used to obtain a meta-policy.

$$r(t) = \begin{cases} \alpha_1 r_1(t) + \cdots + \alpha_M r_M(t), & \sum_{m=1}^{M}\sum_{l=1}^{L} B_{l,m,n}(t) \le N_B^n \ \ and \ \ \sum_{m=1}^{M}\sum_{l=1}^{L} F_{l,m,n}(t) \le N_F^n, \quad n = 1, 2, \ldots, N \\ -1, & \sum_{m=1}^{M}\sum_{l=1}^{L} B_{l,m,n}(t) > N_B^n \ \ or \ \ \sum_{m=1}^{M}\sum_{l=1}^{L} F_{l,m,n}(t) > N_F^n, \quad n = 1, 2, \ldots, N \end{cases} \tag{20}$$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

In an MAML algorithm [28], a meta-policy's parameters are updated according to the parameters of the optimal policy for each objective. An MAML algorithm includes two loops: inner loop and outer loop. In the inner loop, the algorithm trains the parameters of a policy for a specific objective to find an optimal policy, where a gradient descent algorithm is used to minimize the loss function of the QMIX-based SLS algorithm. In the outer loop, the parameters of a meta-policy are updated through calculating the gradient of the parameters for each optimal policy.

b) MAML-based initialization algorithm: The MAML-based algorithm consists of three phases: adaptation phase, meta-policy training phase, and fine tuning phase.

In the adaptation phase, a set of weight vectors $\mathcal{A} = [\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \ldots, \boldsymbol{\alpha}_I], i = 1, 2, \ldots, I$, is given, where I denotes the number of elements in $\mathcal{A}$ and each vector contains a set of weight coefficients, i.e., $\boldsymbol{\alpha}_i = [\alpha_{i,1}, \alpha_{i,2}, \ldots, \alpha_{i,M}]$. For each weight vector, the objective of SP1 is to obtain an optimal policy to perform periodic SLS with the weight coefficients in the vector. To obtain the optimal policy, the parameters of the policy need to be updated iteratively through minimizing the loss function in Eq. (14), i.e.,

$$\boldsymbol{\theta}_i^* = \underset{\boldsymbol{\theta}_i}{\operatorname{argmin}} L(\boldsymbol{\theta}_i | \boldsymbol{\alpha}_i), \tag{22}$$

where $\boldsymbol{\theta}_i^*$ denotes the parameters of an optimal policy π_i^* and $\boldsymbol{\theta}_i$ denotes the parameters of a policy π_i in training.

In the meta-policy training phase, a meta-policy is trained by aggregating the parameters of the optimal policies π_i^* obtained in the previous phase to obtain the optimal parameters of the meta-policy, i.e.,

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}_i}{\operatorname{argmin}} \sum_i L(\boldsymbol{\theta}_i | \boldsymbol{\alpha}_i), \tag{23}$$

where $\boldsymbol{\theta}^*$ denotes the parameters of the meta policy.

In the fine-tuning phase, for the objective of SP1 with a new weight vector, the parameters of the meta-policy can be used as the initial parameters of the agent networks, mixing networks, and hyper-networks used in the QMIX-based algorithm. Then, an optimal policy for the objective can be obtained by finely tuning the initial policy iteratively using the QMIX-based algorithm. A new weight vector will not change over time once it is set in the phase.

The aim of the MAML-based algorithm is to provide an initial optimal policy for the QMIX-based algorithm. The training process of the MAML-based algorithm to obtain an initial optimal policy is based on the previous policies obtained by the QMIX-based algorithm. Thus, the training interval of the MAML-based algorithm is larger than that of the QMIX-based algorithm and the two algorithms are coupled. The pseudo codes of the MAML-based initialization algorithm are given in Algorithm 2. D_i denotes a batch of experiences sampled from the experience pool corresponding to the i-th weight vector.

Algorithm 2 MAML-Based Initialization Algorithm

Input: a set of weight vectors $\boldsymbol{\alpha} = [\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \ldots, \boldsymbol{\alpha}_I]$

```
Randomly initialize a meta-policy;
while training a meta-policy do
    for each weight vector α_i do
        Sample experiences D_i following π_i;
        Update π_i with D_i according to Eq. (22);
        Update the meta-policy according to Eq. (23);
        Set the meta-policy to the initial policy of the QMIX-
        based SLS algorithm;
```

C. PPO-Based DRA Algorithm

A PPO algorithm [29] is a policy-based optimization algorithm, which aims to obtain an optimal policy by training a couple of neural networks. Specifically, it uses an actor network and a critic network to perform action selection and policy evaluation, respectively. The actor network is used to update the parameters of a policy probability distribution with the direction suggested by the critic network and select the best action. However, a policy-based algorithm is often unstable because the magnitude of a gradient step in the parameter space of the algorithm is usually not proportional to the magnitude of a gradient step in the policy space of the algorithm. For a PPO algorithm, the objective function is clipped to avoid having a too large policy update by imposing a constraint on the difference between the current policy and the old one. Thus, a PPO algorithm can achieve better stability and improve the efficiency of training.

SP2 can also be considered as a Markov decision problem with imperfect information because users' computing tasks to arrive in a service-slice instance are unknown. According to [30], a policy-based algorithm can better explore random policies and obtain an optimal policy. Therefore, a PPO-based algorithm is very suitable for being used to perform resource allocation in each service-slice instance.

The PPO-based DRA algorithm consists of two components: spectrum resource allocation (SRA) algorithm and computing resource allocation (CRA) algorithm.

1) PPO-Based SRA Algorithm: In the SRA algorithm, the agent, state, action and reward function are defined as follows:

Agent: An agent is a BS in a service-slice instance.

State: The state of the algorithm is represented by a set of parameters, i.e.,

$$s_{l,m,n}^{BS}(t) = \{\boldsymbol{H}_{l,m,n}(t), L_k(t), X_k(t), \tau_{l,max}\}, \tag{24}$$

where $\boldsymbol{H}_{l,m,n}(t)$ denotes a set of values indicating the releasing time of each SRB on the n-th BS in slice instance $I_{ss}^{l,m}$. Note that the releasing time of an SRB is the interval from the current instant to the instant the SRB becomes idle.

Action: The action of an agent is to allocate one or more SRBs for a computing task, i.e.,

$$a_{l,m,n}^{BS}(t) = \{\widetilde{B}_{l,m,n}^{k}(t)\}, \tag{25}$$

where $\widetilde{B}_{l,m,n}^{k}(t)$ denotes a value following the policy probability distribution with the parameters updated by the actor network, which could be a decimal. Since the number of SRBs allocated to the computing task can only be an integer, $\widetilde{B}_{l,m,n}^{k}(t)$ should be rounded to an integer, which is used as the number of SRBs allocated to the computing task.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

Reward function: The reward function of the algorithm is defined as

$$r_{l,m,n}^{BS}(t) = \begin{cases} \frac{\tau_{l,max}-\tau_{k,tran}(t)-\tau_{k,tranp}(t)}{\tau_{l,max}}, \\ \quad \tau_{k,tran}(t)+\tau_{k,tranp}(t) \le \tau_{l,max} \\ -1, \\ \quad \tau_{k,tran}(t)+\tau_{k,tranp}(t) > \tau_{l,max}. \end{cases} \tag{26}$$

According to Eq. (26), for computing task $u_k^l(t)$, if the sum of its transmission delay and transmission queuing delay does not exceed the maximum tolerant service delay, the reward function is a positive value, which indicates a successful transmission; otherwise, the reward function is -1, indicating a penalty for an unsuccessful transmission.

2) PPO-Based CRA Algorithm: In the CRA algorithm, the agent, state, action and reward function are defined as follows:

Agent: An agent is an edge server in a service-slice instance.

State: The state of the algorithm is given by

$$s_{l,m,n}^{ES}(t) = \{\boldsymbol{G}_{l,m,n}(t), L_k(t), X_k(t), \tau_{l,max}\}, \tag{27}$$

where $\boldsymbol{G}_{l,m,n}(t)$ denotes a set of values indicating the releasing time of each CRB on the *n*-th edge server in slice instance $I_{ss}^{l,m}$.

Action: The action of an agent is to allocate one or more CRBs to a computing task, i.e.,

$$a_{l,m,n}^{ES}(t) = \{\widetilde{F}_{l,m,n}^{k}(t)\}. \tag{28}$$

where $\widetilde{F}_{l,m,n}^{k}(t)$ denotes a continuous value like $\widetilde{B}_{l,m,n}^{k}(t)$. Similar to the PPO-based SRA algorithm, $\widetilde{F}_{l,m,n}^{k}(t)$ should be rounded to an integer, which is used as the number of CRBs allocated to the computing task.

Reward function: The reward function of the algorithm is defined as

$$r_{l,m,n}^{ES}(t) = \begin{cases} \frac{\tau_{l,max}-\tau_k(t)}{\tau_{l,max}}, & \tau_k(t) \le \tau_{l,max} \\ -1, & \tau_k(t) > \tau_{l,max} \end{cases}. \tag{29}$$

According to Eq. (29), for computing task $u_k^l(t)$, if its service delay does not exceed the maximum tolerant service delay, the reward function is a positive value, indicating a successful processing; otherwise, the reward function is a negative value, indicating a penalty for an unsuccessful processing.

The training process of the PPO-based SRA/CRA algorithm consists of a number of iterations. In each iteration, an agent first completes an episode containing N_{step}^P steps. At each step, an agent first obtains the current state, then chooses an action according to the state, executes the chosen action, and obtains a reward. When the episode is completed, the algorithm will first estimate the advantage value $\widehat{A}(t)$ given by

$$\widehat{A}(t) = \sum_{t'>t} \left[\gamma^{t'-t} r(t') - V_{\phi_p}(s(t))\right], \tag{30}$$

where $\widehat{A}(t)$ denotes an advantage function at step t, in which t' denotes a step after t and $V_{\phi_p}(s(t))$ denotes a value function of $s(t)$. Then, it will update its actor network and critic network according to the following equations, respectively,

$$J_{PPO}(\theta_p) = \widehat{E}_t[\min(\psi_t(\theta_p)\widehat{A}_t, clip(\psi_t(\theta_p), 1-\varepsilon, 1+\varepsilon)\widehat{A}_t)], \tag{31}$$

$$L(\phi_p) = -\sum_{t=1}^{N_{step}^P} \sum_{t'>t} \left[\gamma^{t'-t} r(t') - V_{\phi_p}(s(t))\right]^2, \tag{32}$$

where $J_{PPO}(\theta_p)$ denotes the objective function of the PPO algorithm, in which ε is a clipping parameter, θ_p denotes a set of parameters in the actor network, and $clip(\psi_t(\theta_p), 1-\varepsilon, 1+\varepsilon)$ denotes a clip on a policy probability ratio $\psi_t(\theta_p)$; and $L(\phi_p)$ denotes the loss function of the PPO algorithm [29], in which ϕ_p denotes a set of parameters in the critic network. The pseudo codes of the training processes of the two algorithms are described in Algorithm 3, where N_{epis}^P denotes the number of episodes and $\pi(\theta_p)$ is a policy with θ_p.

Algorithm 3 PPO-Based SRA/CRA Algorithm

1: Initialize an actor network and a critic network;
2: **for** $t = 1$ to N_{epis}^P
3: Execute a policy $\pi(\theta_p)$ for N_{step}^P steps and collect $(s_{l,m,n}^{BS}(t), a_{l,m,n}^{BS}(t), r_{l,m,n}^{BS}(t), s_{l,m,n}^{BS}(t+1))/(s_{l,m,n}^{ES}(t), a_{l,m,n}^{ES}(t), r_{l,m,n}^{ES}(t), s_{l,m,n}^{ES}(t+1))$;
4: Estimate the advantage of the algorithm according to $\widehat{A}(t) = \sum_{t'>t}[\gamma^{t'-t} r(t') - V_{\phi_p}(s(t))]$;
5: Update the actor network according to $J_{PPO}(\theta_p) = \widehat{E}_t[min(\psi_t(\theta_p)\widehat{A}_t, clip(\psi_t(\theta_p), 1-\varepsilon, 1+\varepsilon)\widehat{A}_t)]$;
6: Update the critic network according to $L(\phi_p) = -\sum_{t=1}^{N_{step}^P}\sum_{t'>t}[\gamma^{t'-t} r(t') - V_{\phi_p}(s(t))]^2$;

In the PPO-based SRA/CRA algorithm, the actor network and critic network used are trained periodically using previous experiences. Once a computing task arrives in a service-slice instance, the trained actor network and critic network in the PPO-based SRA algorithm and those in the PPO-based CRA algorithm are used to allocate spectrum resources and computing resources for the computing task, respectively.

D. Computational Complexity

Assume that the average number of neurons at each layer in a neural network is N_{nero} and the number of episodes is N_{epis}. The computational complexity for training the neural network is $O((N_{nero})^2 \cdot N_{epis})$. Thus, the computational complexity for training the neural networks used the QMIX-based SLS algorithm is $O((N \cdot N_{nero}^Q)^2 \cdot N_{epis}^Q)$. The computational complexity for training the neural networks used in the PPO-based SRA/CRA algorithm is $O((N_{nero}^P)^2 \cdot N_{epis}^P)$. The computational complexity for training the neural networks used in the MAML-based Initialization Algorithm is $O((N \cdot N_{nero}^Q)^2 \cdot N_{epis}^Q \cdot N_{epis}^M)$, where N_{epis}^M denotes the number of episodes of the MAML-based algorithm.

V. Performance Evaluation

In this section, we evaluate the performance of the proposed RL-S&A algorithm through simulation experiments.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION PARAMETERS

Parameter	Value
Number of edge nodes (N)	5
Number of network-slice instances (M)	3
Total number of SRBs in each edge node (N_B^n)	50
Total number of CRBs in each edge node (N_C^n)	50
Transmission power ($p_k(t)$)	20dBm [6]
Noise power (σ^2)	-100dBm [6]
CPU frequency of a CRB	1GHz [6]
Bandwidth of a SRB	1MHz [6]
Batch size	32
Learning rate (α)	0.01
Discount factor (γ)	0.1
Experience pool size	500

We first evaluate the performance of the PPO-based resource allocation algorithm (PPO) in resource allocation by comparing the algorithm with a random resource allocation algorithm (Random) and an adaptive resource allocation algorithm (Adaptive). The random resource allocation algorithm randomly allocates a certain number of SRBs and CRBs for a user's computing task. The adaptive resource allocation algorithm allocates a certain number of SRBs and CRBs according to the data size and computation intensity of a user's computing task. We then evaluate the performance of the proposed RL-S&A algorithm by comparing the algorithm with a fixed S&A algorithm (FIX), an adaptive S&A algorithm (ADP), and a DQN and PPO based S&A algorithm (DQN-PPO). The fixed algorithm performs periodic SLS and DRA with a fixed number of SRBs and CRBs. The adaptive algorithm performs periodic SLS according to the successful processing ratios of users' computing tasks, and performs DRA according to the data size and computation intensity of a user's computing task. The DQN and PPO based algorithm performs periodic SLS using a DQN algorithm and DRA using the PPO-based algorithm. The DQN algorithm uses M DQNs and the agent of a DQN is the virtual controller of a network-slice instance, which performs SLS in the network-slice instance.

In evaluation, we use the successful processing ratio of users' computing tasks on a BS in a service-slice instance and a weighted average of the successful processing ratios of users' computing tasks in all network-slice instances as the performance metrics. The simulation experiments were conducted using Python, and PyTorch was used to implement the construction and training of neural networks. In the simulation experiments, we assume that the arrivals of users' computing tasks in a service-level instance follow a Poisson process. The main simulation parameters used in the experiments are given in Table I.

A. Performance of the PPO-Based DRA Algorithm

Fig. 3 shows the successful processing ratios on a BS in a service-slice instance vs the arrival rate of users' computing tasks with the PPO-based algorithm, the adaptive algorithm, and the random algorithm, respectively. It is seen that as the arrival rate increases, the successful processing ratios with all the three algorithms gradually decrease. This is because when the arrival rate becomes larger, more computing tasks will arrive at a BS and a computing task may experience a longer waiting time before being processed, resulting in a lower proportion of users' computing tasks that meet the maximum tolerant service delay. On the other hand, the successful processing ratio with the PPO-based algorithm is larger than that with the adaptive algorithm and that with the random algorithm. This is because the PPO-based algorithm can allocate spectrum and computing resources more efficiently according to a computing task's demand and the current resource status on the BS to improve the successful processing ratio.

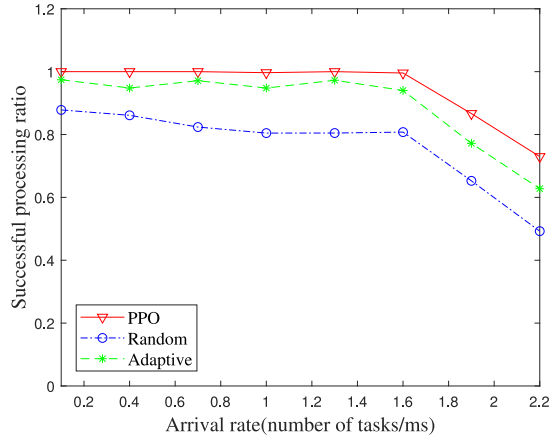

Fig. 3. Successful processing ratio vs arrival rate.

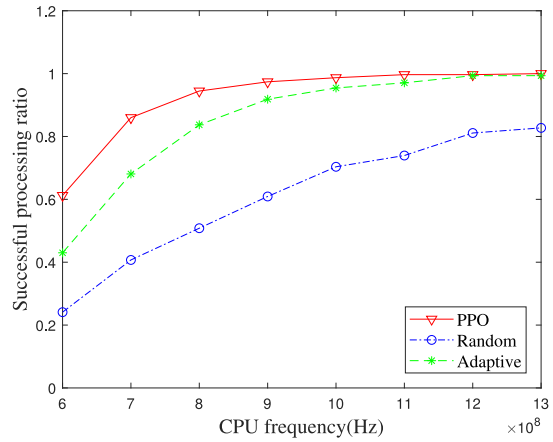

Fig. 4. Successful processing ratio vs CPU frequency.

Fig. 4 shows the successful processing ratios on a BS in a service-slice instance vs the CPU frequency of a CRB with the PPO-based algorithm, the adaptive algorithm, and the random algorithm, respectively. It is seen that as the CPU frequency increases, the successful processing ratios with all the three algorithms increase first and then gradually tend to be stable. This is because when the CPU frequency increases, a computing task has a higher probability to be allocated sufficient computing resources and meet its maximum tolerant service delay. On the other hand, the successful processing ratio with the PPO-based algorithm is larger than that with the

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

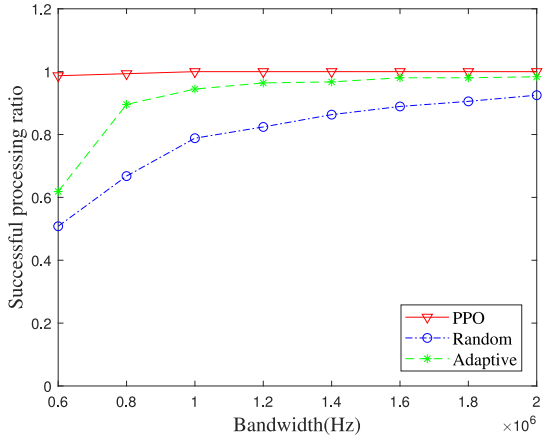

Fig. 5. Successful processing ratio vs bandwidth with the three algorithm.

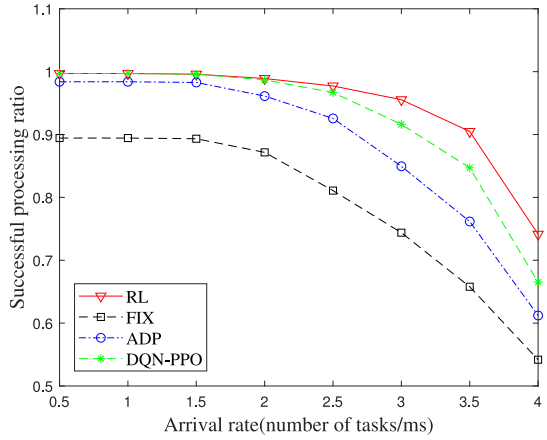

Fig. 7. Successful processing ratio vs arrival rate with the four algorithm.

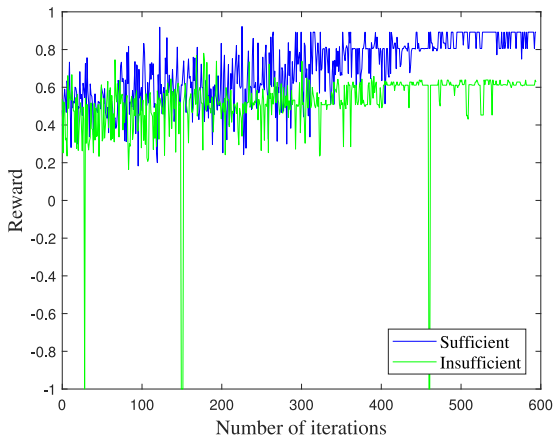

Fig. 6. Reward vs number of iterations.

adaptive algorithm and that with the random algorithm. The reason is the same as that explained in Fig. 3.

Fig. 5 shows the successful processing ratios on a BS in a service-slice instance vs the bandwidth of a SRB with the PPO-based algorithm, the adaptive algorithm and the random algorithm, respectively. It is seen that as the bandwidth increases, the successful processing ratios with all the three algorithms increase first and then gradually tend to be stable. This is because when the bandwidth increases, a computing task has a higher probability to be allocated sufficient spectrum resources and meet its maximum tolerant service delay. On the other hand, the successful processing ratio with the PPO-based algorithm is larger than that with the adaptive algorithm and that with the random algorithm. The reason is the same as that explained in Fig. 3.

B. Performance of the RL-S&A Algorithm

Fig. 6 shows the reward value with the QMIX-based SLS algorithm vs the number of iterations when the spectrum and computing resources in the network are sufficient and insufficient, respectively. It is seen that as the number of iterations increases, the reward values in both the cases of sufficient resources and insufficient resources gradually converge, indicating the convergence of the algorithm. On the other hand, when the resources in the network are sufficient, the algorithm can achieve a larger reward value, which means a higher successful processing ratio.

Fig. 7 shows the successful processing ratios vs the arrival rate of users' computing tasks with the RL-RSA algorithm, the DQN-PPO algorithm, the FIX algorithm and the ADP algorithm, respectively. It is seen that as the arrival rate increases, the successful processing ratios with all the four algorithms gradually decrease. This is because when the arrival rate becomes larger, more computing tasks will arrive at a BS and a computing task may experience a longer waiting time before being processed, resulting in a larger service delay and a lower successful processing ratio. On the other hand, the successful processing ratio with the RL-S&A algorithm and that with the DQN-PPO algorithm are larger than that with the ADP algorithm and that with the FIX algorithm. This is because the RL-S&A algorithm and the DQN-PPO algorithm can perform periodic SLS to increase the resources in those service-slice instances to improve the successful processing ratio. Moreover, when the arrival rate is smaller, the successful processing ratio with the RL-S&A algorithm is almost equal to that with the DQN-PPO algorithm. This is because in this case the resources in the network are sufficient. When the arrival rate becomes larger, the successful processing ratio with the RL-S&A algorithm becomes larger than that with the DQN-PPO algorithm. This is because the RL-S&A algorithm performs SLS in each network-slice instance jointly, while the DQN-PPO algorithm performs SLS in each network-slice instance separately.

Fig. 8 shows the successful processing ratios vs the CPU frequency of a CRB with the RL-S&A algorithm, the DQN-PPO algorithm, the FIX algorithm and the ADP algorithm, respectively. It is seen that as the CPU frequency increases, the successful processing ratios with all the four algorithms gradually increase. This is because as the CPU frequency increases, a computing task can be processed and completed more quickly. On the other hand, the successful processing ratio with the RL-S&A algorithm and that with the DQN-PPO algorithm are larger than that with the ADP algorithm and that with the FIX algorithm. Moreover, the RL-S&A algorithm

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

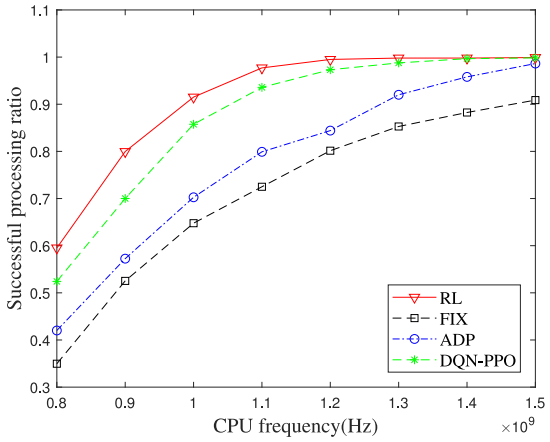

Fig. 8. Successful processing ratio vs CPU frequency with the four algorithm.

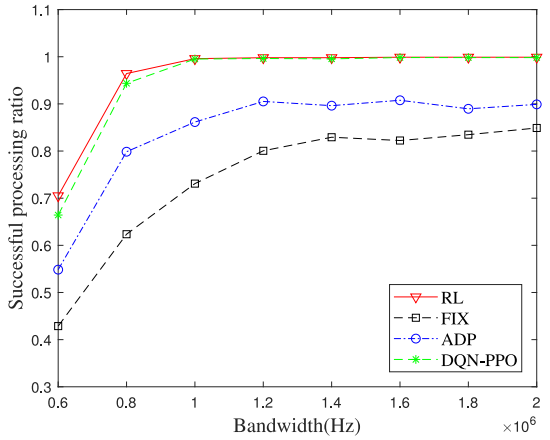

Fig. 9. Successful processing ratio vs bandwidth with the four algorithm.

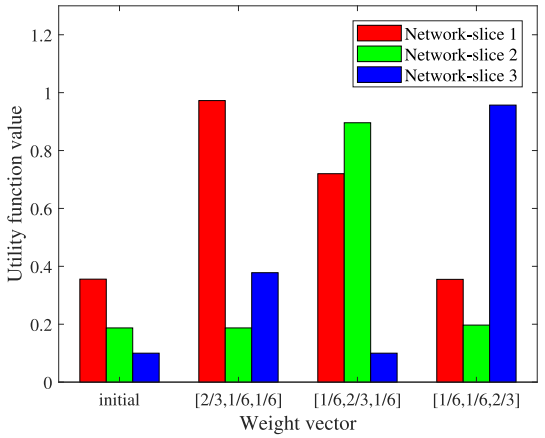

Fig. 10. Utility function value vs weight vector.

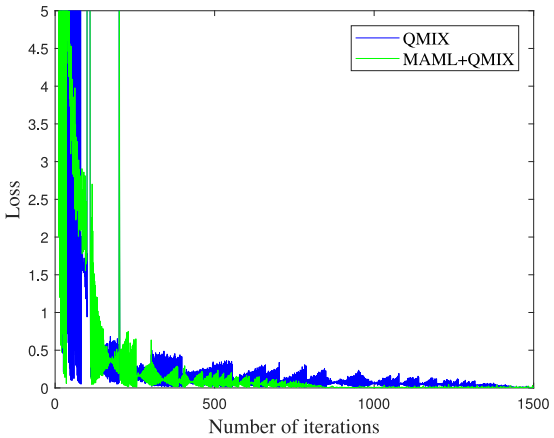

Fig. 11. Loss value vs number of iterations.

outperforms the DQN-PPO algorithm. The reasons are similar to those explained in Fig. 7.

Fig. 9 shows the successful processing ratios vs the bandwidth of a SRB with the RL-S&A algorithm, the DQN-PPO algorithm, the FIX algorithm and the ADP algorithm, respectively. It is seen that as the bandwidth increases, the successful processing ratios with all the four algorithms gradually increase and tend to be stable. This is because when the bandwidth increases, a computing task can be transmitted to a BS more quickly. On the other hand, the successful processing ratio with the RL-S&A algorithm and that with the DQN-PPO algorithm is larger than that with the ADP algorithm and that with the FIX algorithm. Moreover, the RL-S&A algorithm outperforms the DQN-PPO algorithm. The reasons are similar to those explained in Fig. 7.

Fig. 10 shows the utility function values of three network-slice instances vs different weight vectors for the objective in the formulated problem with the QMIX-based SLS algorithm. The three weight vectors correspond to three different objective functions of the formulated problem. It is seen that for each weight vector, the three values represent the utility function values corresponding to the three network-slice instances after SLS is performed. The three values with "initial" represent the utility function values of the three network-slice instances before SLS is performed. It is seen that for a larger weight, the corresponding network-slice instance obtains a larger utility function value. This is because for a larger weight, the central controller of the QMIX-based SLS algorithm will perform SLS in the corresponding network-slice instance with a higher probability, resulting a larger utility function value.

Fig. 11 shows the loss value vs the number of iterations with the QMIX-based algorithm using a set of optimal initialization parameters and that not using a set of optimal initialization parameters. It is seen that as the number of iterations increases, the loss values with both the two algorithms gradually decrease and tends to be stable, indicating the convergence of the algorithms. On the other hand, the loss value with the QMIX-based algorithm using optimal initialization parameters decreases more quickly, indicating faster convergence and the effect of the MAML-based initialization algorithm.

VI. Conclusion

In this paper, we considered the S&A problem in an MEC network. We introduced a two-level slicing model and formulated the problem as a long-term optimization problem with an objective to maximize a weighted average of the successful processing ratios of users' computing tasks in each network slice instance by performing network slicing and

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

resource allocation, while satisfying the service requirements of users' computing tasks. To solve the formulated problem, the problem was decomposed into the SLS sub-problem and the DRA sub-problem, and a RL-S&A algorithm was proposed to solve the two sub-problems by using a QMIX-based SLS algorithm and a PPO-based DRA algorithm, respectively. To reduce the training time of an optimal policy to perform periodic SLS, an MAML-based initialization algorithm was further introduced to obtain a meta-policy to initialize the neural networks used in the QMIX-based algorithm. Extensive simulation results show that the proposed RL-S&A algorithm outperforms the three benchmark algorithms in terms of the successful processing ratio of users' computing tasks in the network.

REFERENCES

[1] C.-F. Liu, M. Bennis, M. Debbahm, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Trans. Commun.*, vol. 67, no. 6, pp. 4132–4150, Jun. 2019.

[2] A. Abouaomar, S. Cherkaoui, Z. Mlika, and A. Kobbane, "Resource provisioning in edge computing for latency-sensitive applications," *IEEE Internet Things J.*, vol. 8, no. 14, pp. 11088–11099, Jul. 2021.

[3] W. Zhuang, Q. Ye, F. Lyu, N. Cheng, and J. Ren, "SDN/NFV-empowered future IoV with enhanced communication, computing, and caching," in *Proc. IEEE*, vol. 108, no. 2, pp. 274–291, Feb. 2020.

[4] L. Cominardi, T. Deiss, M. Filippou, V. Sciancalepore, F. Giust, and D. Sabella, "MEC support for network slicing: Status and limitations from a standardization viewpoint," *IEEE Commun. Stand. Mag.*, vol. 4, no. 2, pp. 22–30, Jun. 2020.

[5] S. D. A. Shah, M. A. Gregory, and S. Li, "Cloud-native network slicing using software defined networking based multi-access edge computing: A survey," *IEEE Access*, vol. 9, pp. 10903–10924, 2021.

[6] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[7] K. Guo, R. Gao, W. Xia, and T. Q. S. Quek, "Online learning based computation offloading in MEC systems with communication and computation dynamics," *IEEE Trans. Commun.*, vol. 69, no. 2, pp. 1147–1162, Feb. 2021.

[8] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 881–892, Sep. 2021.

[9] J. Wang, L. Zhao, J. Liu, and N. Kato, "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1529–1541, Jul.–Sep. 2021.

[10] J. Wang, Y. Wang, P. Cheng, K. Yu, and W. Xiang, "DDPG-based joint resource management for latency minimization in NOMA-MEC networks," *IEEE Commun. Lett.*, vol. 27, no. 7, pp. 1814–1818, Jul. 2023.

[11] Z. Sharif, L. T. Jung, I. Razzak, and M. Alazab, "Adaptive and priority-based resource allocation for efficient resources utilization in mobile-edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3079–3093, Feb. 2023.

[12] Q. Ye, W. Shi, K. Qu, H. He, W. Zhuang, and X. Shen, "Joint RAN slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 272–288, 2021.

[13] Q. Ye, W. Zhuang, S. Zhang, A.-L. Jin, X. Shen, and X. Li, "Dynamic radio resource slicing for a two-tier heterogeneous wireless network," *IEEE Trans. Veh. Technol.*, vol. 67, no. 10, pp. 9896–9910, Oct. 2018.

[14] A. Ndikumana, K. K. Nguyen, and M. Cheriet, "Two-level closed loops for RAN slice resources management serving flying and ground-based cars," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1184–1198, Jun. 2023.

[15] A. Awad Abdellatif, A. Abo-Eleneen, A. Mohamed, A. Erbad, N. V. Navkar, and M. Guizani, "Intelligent-slicing: An AI-assisted network slicing framework for 5G-and-beyond networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1024–1039, Jun. 2023.

[16] H. Zhang, G. Pan, S. Xu, S. Zhang, and Z. Jiang, "A hard and soft hybrid slicing framework for service level agreement guarantee via deep reinforcement learning," in *Proc. IEEE 95th Veh. Technol. Conf. (VTC)*, Helsinki, Finland, 2022, pp. 1–5.

[17] J. Feng, Q. Pei, F. R. Yu, X. Chu, J. Du, and L. Zhu, "Dynamic network slicing and resource allocation in mobile edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7863–7878, Jul. 2020.

[18] Y. Hua, R. Li, Z. Zhao, H. Zhang, and X. Chen, "GAN-based deep distributional reinforcement learning for resource management in network slicing," in *Proc. IEEE Global Commun. Conf.*, Waikoloa, HI, USA, 2019, pp. 1–6.

[19] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4005–4018, Jun. 2019.

[20] T. Dong et al., "Intelligent joint network slicing and routing via GCN-powered multi-task deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 2, pp. 1269–1286, Jun. 2022.

[21] S. R. A. Sebakara, G. Sun, G. O. Boateng, B. Mareri, R. Ou, and G. Liu, "SNAF: DRL-based interdependent E2E resource slicing scheme for a virtualized network," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9069–9084, Jul. 2023.

[22] T. Mohammed, B. Jedari, and M. Di Francesco, "Efficient and fair multi-resource allocation in dynamic fog radio access network slicing," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 24600–24614, Dec. 2022.

[23] L. Yu, S. Jiang, J. Zheng, F. Yan, and S. Zhao, "A DQN-based joint computing offloading and resource allocation algorithm for MEC networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Rome, Italy, 2023, pp. 2553–2558.

[24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.

[25] T. Rashid, M. Samvelyan, C. D. Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, 2018, pp. 1–10.

[26] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning," in *Proc. 17th Int. Conf. Autonom. Ag. MultiAgent Syst. (AAMAS)*, 2018, pp. 2085–2087.

[27] A. Marcin et al., "Learning to learn by gradient descent by gradient descent," in *Proc. 30th Annu. Conf. Neural Inf. Process. Syst. (NIPS)*, 2016, pp. 1–9.

[28] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1–10.

[29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv.1707.06347*.

[30] N. Heess et al., "Emergence of locomotion behaviours in rich environments," 2017, *arXiv.1707.02286*.

Shurui Jiang received the B.E. degree in information engineering from the Nanjing University of Aeronautics and Astronautics, China, in 2021. She is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, Southeast University, China. She has authored or coauthored six journal and conference papers. Her research interests include resource management in mobile-edge computing networks and application of machine learning in mobile communications and networks.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.

Jun Zheng (Senior Member, IEEE) received the Ph.D. degree in electrical and electronic engineering from The University of Hong Kong, Hong Kong, in 2000. He is currently a Full Professor with the School of Information Science and Engineering, Southeast University (SEU), China, and also a member with Purple Mountain Laboratories, China. Before joining SEU in 2008, he was with the School of Information Technology and Engineering, University of Ottawa, Canada. He has coauthored (first author) two books published by Wiley-IEEE Press, and has coauthored nearly 200 technical papers in refereed journals and peer-reviewed conference proceedings. His current research interests include mobile communication networks, vehicular ad hoc networks, and UAV-connected networks, focused on network architectures and protocols. He is a recipient of 2021 IEEE ComSoc Communications Software Technical Committee "Technical Achievement Award" and 2023 IEEE ComSoc Communications Switching & Routing Technical Committee "Distinguished Technical Achievement Award". He was a co-recipient of the Best Paper Awards at IEEE ICC 2014 and WCSP 2018. He is currently an editorial board member of several international journals, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY. He has co-edited 14 special issues for different refereed journals and magazines, including *IEEE Communications Magazine*, IEEE NETWORK, and IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, all as a Lead Guest Editor. He has served as the Founding General Chair of AdHocNets'2009, the General Chair of AccessNets'2007, AdHocNets'2018/2019, and ICNC 2024, and a TPC or Symposium Co-Chair for a number of international conferences and symposia, including IEEE ICC'2009/2011/2015/2021 and GLOBECOM'2008/2010/2012/2018/2019. He has also served as a TPC member for a number of international conferences and symposiums. He is a senior member of the IEEE Communications Society and IEEE Vehicular Technology Society. He serves as the Chair of IEEE Vehicular Technology Society Nanjing Chapter, and was the Chair of IEEE ComSoc Communications Switching and Routing Technical Committee.

Feng Yan (Member, IEEE) received the B.S. degree in electrical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2005, the M.S. degree in electrical engineering from Southeast University, Nanjing, China, in 2008, and the Ph.D. degree in electrical engineering from Telecom ParisTech, Paris, France, in 2013. From November 2013 to April 2015, he was a Postdoctoral Researcher with Telecom Bretagne, Rennes, France. He is currently an Associate Professor with the National Mobile Communications Research Laboratory, Southeast University. His current research interests are in the areas of wireless sensor networks, UAV networks, and IoT networks.

Shuyuan Zhao received the Ph.D. degree in information engineering from Xi'an Jiaotong University, Xi'an, China, in 2020. He is working as a Postdoctoral Researcher with Zhejiang Laboratory, Hangzhou, China. He has published seven journal papers and three conference papers. His current research interests include mobile-edge computing, vehicular networks, and deep reinforcement learning.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on February 25,2026 at 09:17:36 UTC from IEEE Xplore. Restrictions apply.